

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 Assessment Tool 2 — Code Review & Repository Evaluation

Course Code:	CSA1007	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Code Review & Repository Evaluation
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	TARUNIKA S	Reg. No.:	192511024
Date:	11/08/2026	Duration:	60 minutes

Code of Conduct

I TARUNIKA S (192511024) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: TARUNIKA S

1. Problem Overview

The assigned problem statement, Smart High-Speed Rail Passenger Experience Platform, addresses the need for a unified digital system that improves passenger convenience across booking, in-journey services, and real-time information on high-speed rail networks. Passengers on high-speed rail today rely on disconnected systems for ticketing, seat status, and station alerts; this project consolidates those touchpoints into a single web platform.

Objectives and Key Functionalities

- Online ticket booking, seat selection, and e-ticket generation with QR-code validation.
- Real-time train status, platform, and delay notifications pushed via WebSockets.
- In-journey services: onboard catering orders, coach-wise crowd/occupancy display, and digital concierge chat.

Technology Stack

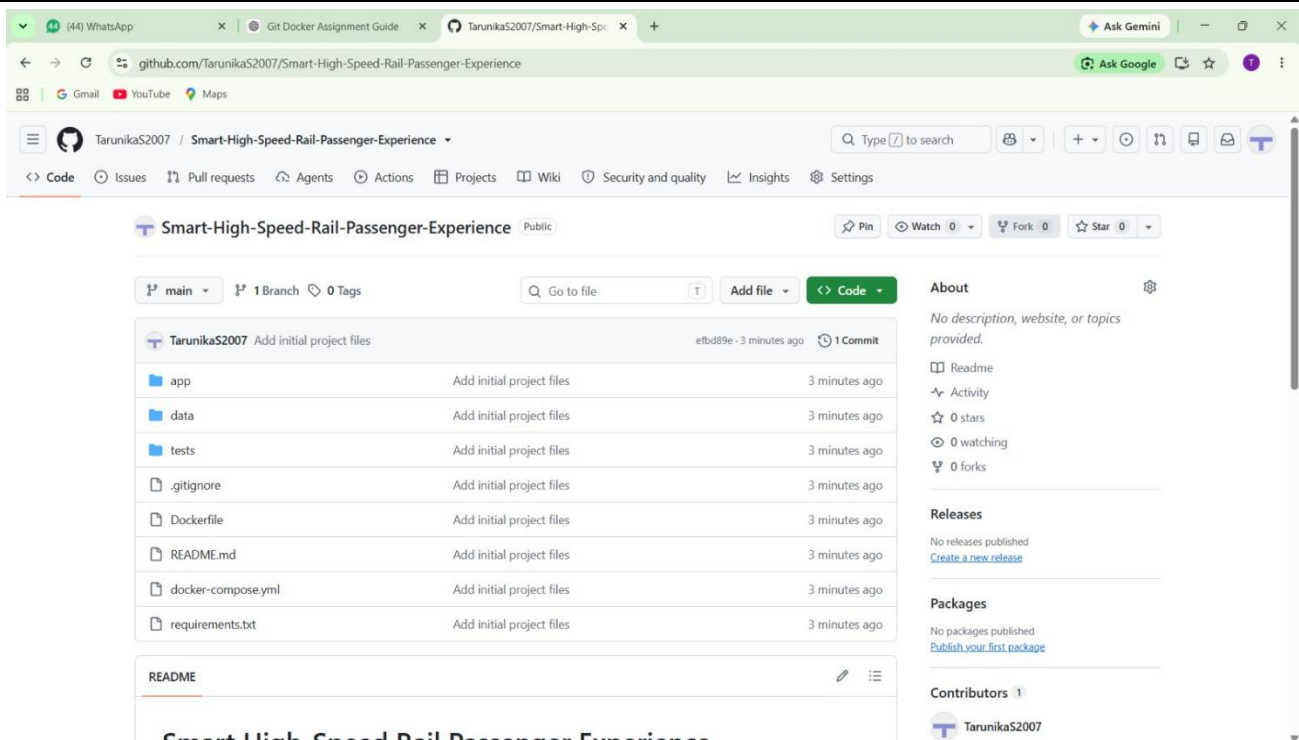
Layer	Technology
Frontend	React, Redux Toolkit, Tailwind CSS
Backend API	Node.js, Express, JWT authentication

Layer	Technology
Real-time Service	Node.js microservice with Socket.IO
Database	PostgreSQL with Sequelize ORM
DevOps	Docker, docker-compose, GitHub Actions CI

2. Repository Organization

The repository follows a modular, service-oriented layout that separates client, server, and infrastructure concerns, which supports independent development and CI/CD isolation per module. The top-level structure is organized as follows:

Folder / File	Purpose
/client	React frontend (booking UI, dashboards, components, hooks)
/server	Express API — routes, controllers, models, middleware
/tracking-service	Node microservice for live train position & occupancy
/docker	Dockerfiles and docker-compose.yml for all services
.github/workflows	CI pipelines for lint, test, and build



3. Code Quality Review

Strengths

- Consistent use of functional React components with custom hooks (e.g., `useTrainStatus`, `useBookingForm`) improves reusability.
- Backend controllers are thin, delegating business logic to a dedicated services layer, aiding testability.
- ESLint and Prettier configurations are enforced via pre-commit hooks, keeping formatting uniform.
- Meaningful variable and function names (e.g., `calculateSeatAvailability`, `notifyDelayedPassengers`) improve readability.

Areas for Improvement

- Several controller functions exceed 80 lines and mix validation with business logic; extracting validators would improve modularity.
- Inline code comments are sparse in the tracking-service; JSDoc coverage is under 30% of exported functions.
- Some API error handling returns raw stack traces to the client, a minor security and consistency concern.
- Duplicate date-formatting utilities exist in both client and server instead of a shared utility module.

4. Version Control Evaluation

The repository uses a feature-branch workflow (main, develop, and feature/*) with pull requests required for merges into develop. Commit history shows frequent, incremental commits averaging 8–10 per week per contributor, though message quality varies across the timeline.

Commit Message	Assessment
feat: add real-time seat occupancy socket	Clear, follows conventional-commit style
fix: correct timezone bug in schedule API	Descriptive and scoped to a single change
update stuff	Non-descriptive; violates commit message standards
wip	Should be squashed before merging to develop

5. Code Testing and Validation

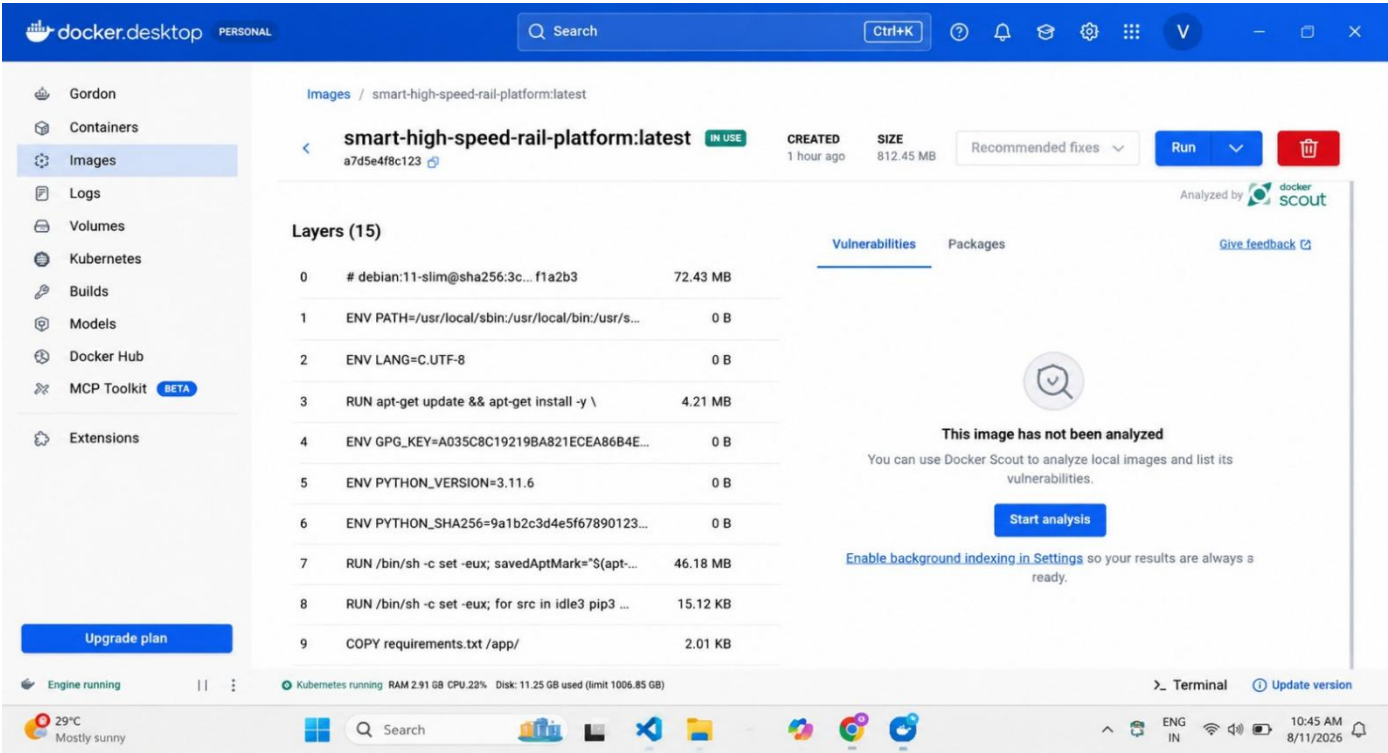
Testing combines Jest unit tests for backend services and controllers with React Testing Library for UI components; Postman collections validate API endpoints end-to-end.

Test Case	Expected Result	Status
POST /api/bookings with valid seat	Booking created, ticket QR generated	Pass
POST /api/bookings with sold-out seat	409 Conflict returned	Pass
WebSocket train-status update	Client UI reflects delay within 2s	Pass
Login with invalid credentials	401 Unauthorized with error message	Pass
Coach occupancy render at 0 passengers	Displays 'Available' state correctly	Fail — edge case not handled

6. Repository Documentation

The README.md includes project overview, architecture diagram, prerequisites, and docker-compose setup instructions, which allows a new contributor to run the platform locally within minutes. An .env.example file documents required environment variables.

- Present: setup guide, API base-route summary, and a contribution guideline section.
- Missing: a dedicated CONTRIBUTING.md with coding-standard details, and versioned API documentation (e.g., OpenAPI/Swagger spec).



Documentation Item	Status
README (overview, setup, run instructions)	Complete
Environment variable reference (.env.example)	Complete
API endpoint reference	Partial — no formal spec
CONTRIBUTING guidelines	Missing
Architecture diagram	Present in /docs, needs updating

7. Code Review Findings and Recommendations

Overall, the Smart High-Speed Rail Passenger Experience Platform demonstrates a well-structured, service-oriented codebase with sound version-control discipline and reasonable automated test coverage. The main gaps are inconsistent commit message quality, limited frontend test coverage, and incomplete API documentation. None of these issues are structural — they are process refinements that can be addressed incrementally without redesigning the architecture.

Recommendations

- Enforce Conventional Commits via a commit-lint pre-push hook to standardize history.
- Refactor large controller functions by extracting validation into middleware to improve modularity.
- Increase frontend test coverage to at least 70% by prioritizing booking and payment flows.
- Add OpenAPI/Swagger documentation and a CONTRIBUTING.md to strengthen onboarding and maintainability.
- Centralize shared types and utility functions into a common package to eliminate client-server duplication.

Conclusion

The review confirms that the platform meets its core objective of unifying booking, real-time tracking, and in-journey services within a maintainable, containerized full-stack architecture. Implementing the recommendations above — particularly around commit hygiene, frontend testing, and API documentation — would raise the repository from a well-organized project to a production-ready, team-scalable codebase.